

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA0211

COURSE NAME: C Programming

COURSE OUTCOME COVERED

CO1: Construct structured C programs using constants, variables, data types, and preprocessor directives to address basic computational tasks. (BL2, BL3)

Assessment Tool Weightage: 40%

QUESTIONS:

Total Marks:100

Annexure A

PSEUDO CODE WRITING

Code of Conduct:

I Y-Hema Sai Lahari ⁽¹⁹²³¹¹⁴⁴⁶⁾ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Y-Hema Sai Lahari

PSEUDO CODE WRITING

Q1. Given pseudocode:
#define DEBUG 0

```
IF DEBUG  
  PRINT intermediate_value  
END IF
```

- Identify error in directive usage
- Correct the logic

Q2. #define PI _____

```
DECLARE radius AS float  
DECLARE area AS float
```

```
INPUT radius
```

```
area = _____
```

```
PRINT area
```

Fill:

- Constant
- Formula

Q3. #define RATE 1.5

```
DECLARE x AS int = 10  
DECLARE result AS float
```

```
result = x * RATE
```

```
PRINT result
```

Questions:

- Output?
- Why is the result float?

Q4. #define MAX 1000

DECLARE temp AS int
INPUT temp

IF temp > MAX
 PRINT "Valid"
ELSE
 PRINT "Invalid"

- Identify logical error
- Suggest correct condition

Q5. Given unordered pseudocode steps:

- DEFINE conversion rate
- PRINT result
- INPUT amount
- CALCULATE converted value

Arrange in correct order

Q6. #define MIN 10
 #define MAX 50

DECLARE value AS int
INPUT value

_____ (value >= MIN AND value <= MAX)
PRINT "Within range"

_____ PRINT "Out of range"

Fill missing control structures

Q7. A billing system includes optional features such as discount calculation and tax computation. Depending on the business requirement, these features may be enabled or disabled at compile time. The program should calculate the total bill amount based on input price and quantity. If discount and tax features are enabled, they should be applied using predefined constants; otherwise, they should be skipped. The system must ensure that floating-point precision is maintained during calculations.

Write pseudocode using preprocessor directives to enable/disable features, define constants for discount and tax rates, and use appropriate data types.

SECRET

Q8. #define TAX 5%

DECLARE price AS int
DECLARE total AS int

total = price + (price * TAX)

PRINT total

- Identify errors
- Correct data types and constant definition

9. A temperature conversion program supports multiple modes such as Celsius to Fahrenheit and Fahrenheit to Celsius. The mode of operation should be selected at compile time using preprocessor directives. The program should use appropriate floating-point data types to ensure accuracy and define conversion formulas using symbolic constants wherever applicable. The final output should be displayed with two decimal precision.

Write pseudocode using preprocessor directives for mode selection, constants for conversion formulas, and proper data types.

10. #define SIZE 5

DECLARE arr[SIZE]
INPUT elements

- Extend pseudocode to:
 - Find sum
 - Compute average

Q1

Error: IF DEBUG is incorrect because #define is a preprocessor directive. It should use a Compile-time condition.

Correct Logic:

```
#define DEBUG 1
#ifdef DEBUG
PRINT intermediate_value
#endif
```

Q2

```
#define PI 3.14
DECLARE radius AS float
DECLARE area AS float
INPUT radius
area = PI * radius * radius
PRINT area
```

Q3

```
#define RATE 1.5
DECLARE X AS int = 10
DECLARE result AS float
result = X * RATE
PRINT result
```

float: Because RATE is a floating point value (1.5), the multiplication produces a float.

Q4

```

# define MAX 1000
DECLARE temp AS int
INPUT temp

IF temp > MAX
    PRINT "Valid"
ELSE
    PRINT "Invalid"

```

Correct condition:

```

IF temp <= MAX
    PRINT "Valid"
ELSE
    PRINT "Invalid"

```

Logical Error: The condition is reversed

Q5

Correct order:

1. DEFINE Conversion rate

2. INPUT amount

3. CALCULATE converted value

4. PRINT result

Q6

```
# define MIN 10
```

```
# define MAX 50
```

```
DECLARE value AS int
```

```
INPUT value
```

```
IF (value >= MIN AND value <= MAX)
```

```
    PRINT "within range"
```

```
ELSE
```

```
    PRINT "out of range"
```

```
ENDIF
```


Missing Control Structures:

Pg: 2

- * IF
- * ELSE
- * ENDIF

```
# define DISCOUNT 10
# define TAX 5
# define ENABLE - DISCOUNT
# define ENABLE - TAX
```

```
DECLARE price, quantity As float
DECLARE total As float
```

```
INPUT price
INPUT quantity
```

```
total = price * quantity
```

```
# if def ENABLE - DISCOUNT
total = total - (total * DISCOUNT / 100)
# endif
```

```
# ifdef ENABLE - TAX
total = total + (total * TAX / 100)
# endif
```

```
PRINT total
```

Errors:

- * #define TAX 5% is invalid
- * price and total should be float

define TAX 0.05
DECLARE price AS float
DECLARE total AS float
INPUT price
total = price + (price * TAX)
PRINT total

Q9
define C-TO-F
DECLARE temp AS float
DECLARE resut AS float
INPUT temp
ifdef C-TO-F
result = (temp * 9/5) + 32

endif

ifdef F-TO-C
result = (temp - 32) * 5/9

endif

PRINT result

Q10
define SIZE 5
DECLARE arr [SIZE]
DECLARE Sum AS int = 0
DECLARE avg AS float

FOR i = 0 TO SIZE - 1

INPUT arr[i]

Sum = Sum + arr[i]

END FOR

avg = Sum / SIZE

PRINT Sum, PRINT avg